# Orquestación de Agentes Autónomos con n8n

## Procesamiento del Lenguaje Natural — Codespaces self-hosted

Prof. Lucas Miguel Iturriago Salas

Universidad Nacional de Colombia
Dept. Ing. Eléctrica, Electrónica y Computación
Primer Semestre – 2026

# Estructura de la Sesión

1. Sistemas de Agentes Orientados a Eventos
2. Inicialización del Agente y Paradigma ReAct
3. Grounding y Herramientas Dinámicas
4. Despliegue en Producción y MLOps **(en Codespaces)**

# De la Inferencia Secuencial a los Grafos de Ejecución

En sesiones previas operamos bajo inferencia estrictamente secuencial:

$$S_t = f(S_{t-1}, y_t)$$

Al escalar a Agentes Autónomos pasamos a un **Grafo Dirigido de Ejecución**. El agente es un sistema reactivo conducido por eventos. Cada nodo muta y transporta un payload JSON.

# El Mindset de Nodos en la Arquitectura de Software

**EDA:** el LLM actúa como motor de enrutamiento dinámico sobre conectores I/O.
Sustituimos canalizaciones rígidas de Python por un lienzo visual serverless.

Anatomía de un evento en n8n:

```
[ { "json": { "meta": { "timestamp": "..." } } } ]
```

# El Paradigma ReAct: Razonamiento y Acción

1. **Thought:** razonamiento latente $t_t \in T$.
2. **Action:** invoca herramienta $a_t \in A$ con esquema JSON estricto.
3. **Observation:** el entorno devuelve $o_t \in O$.

$$\text{Prompt}_{t+1} = [\text{Prompt}_t \parallel t_t \parallel a_t \parallel o_t]$$

# El Nodo 'AI Agent' y el Desacoplamiento de Capas

- **Cómputo (Model sub-node):** inferencia y T= 0 vía OpenRouter (`meta-llama/llama-3.3-70b-instruct:free`).
- **Lógica (Agent node):** System Instruction, restricciones operativas.

Beneficio: hot-swap del LLM (Llama $\leftrightarrow$ cualquier modelo `:free` de OpenRouter) sin tocar tools ni personalidad.

# Hiperparámetros Críticos: Temperatura Cero ($T = 0.0$)

$$P(y_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Con $T \to 0$ la distribución colapsa a one-hot sobre el logit mayor: se elimina la entropía. Para invocar herramientas queremos **determinismo**, no creatividad.

# Gestión de Estado: Window Buffer Memory

Webhook/REST es stateless. Conectamos **Window Buffer Memory** (FIFO de ventana $k = 10$): poda tokens antiguos, reduce latencia y evita saturar el contexto.

# Rompiendo el Conocimiento Estático con 'Grounding'

**Grounding:** anclar la generación a fuentes verificables en tiempo real.

- **Local (default Codespaces):** nodo `Spreadsheet File` → `/home/node/files/Clase 10/Inventario Laboratorio UNAL.xlsx`, hoja `Hoja 1`.
- **Demo docente en vivo:** swap a `Google Sheets Tool` (Sheet within document, Get Rows).
- **Telemetría:** `HTTP Request Tool` GET a Open-Meteo (Manizales 5.07, -75.52).

# El Vector de Activación Semántica (Descriptions)

El LLM no conoce URLs ni código interno: lee el campo **Description** del sub-nodo, hace matching de intención y extrae argumentos. Sin buena descripción, no hay invocación.

# Arquitectura Self-Hosted en Codespaces (nuevo)

Sin n8n Cloud. Dos contenedores vía `.devcontainer/docker-compose.yml`:

```
cd .devcontainer
docker compose up -d   # dev + n8n:1.107.0 en :5678
```

- n8n en `http://localhost:5678` (forward `*.app.github.dev`).
- Repo montado RO en `/home/node/files` → inventario sin subir a mano.
- Importar: `Workflows > Import from file > Clase 10/n8n-workflow.json`.
- Credencial OpenRouter con key gratuita de openrouter.ai (modelo `llama-3.3-70b-instruct:free`).
- **Production:** `/webhook/...` con workflow en `Active`. **Test:** `/webhook-test/...` solo con `Execute workflow` abierto.

# Resolución del Error: 'No Session ID found'

Sin chat integrado, la memoria pierde el remitente. Mapeo explícito **Define below**:

- Memoria: `{{ $('Webhook').item.json.body.sessionId }}`
- Prompt: `{{ $json.body.chatInput }}`

Payload raíz del Webhook:

```
{"sessionId": "lucas-unal-001", "chatInput": "Hay osciloscopios?"}
```

# Consumo e Integración desde el Codespace (nuevo)

```
python "Clase 10/Peticion.py" --mensaje "Hay osciloscopios?" --session-id lucas-01
export N8N_WEBHOOK_URL="https://TU-CODESPACE-5678.app.github.dev/webhook/agente-
    logistica-unal"
python "Clase 10/Peticion.py"
```

```
curl -X POST http://localhost:5678/webhook/agente-logistica-unal \
 -H "Content-Type: application/json" \
 -d '{"sessionId":"lucas-01","chatInput":"Hay osciloscopios?"}'
```

Default: `localhost:5678/webhook/agente-logistica-unal`. Ver `Guia_Codespace.md`.

# MLOps Observability y Auditoría de Tokens

Pestaña **Executions**: payload por arista, tokens/min, latencia y determinismo de function calls. ¡Pipeline listo para producción en Codespaces!